

Smart Shiksha: An AI-Powered, Offline-First, Multilingual Learning Platform for Rural India

1st Dr. C. Viji

Associate Professor
Alliance University
Bengaluru, India
viji.c@alliance.edu.in

2nd Yashwanth M Gowda

B Tech CSE AIML
Alliance University
Bengaluru, India
yashugowda8102005@gmail.com

3rd M Numan

B Tech CSE AIML
Alliance University
Bengaluru, India
mnumanvig@gmail.com

4th Narasimha Parashuram Savur

B Tech CSE AIML
Alliance University
Bengaluru, India
narasimhasavur@gmail.com

5th Harish L

B Tech CSE AIML
Alliance University
Bengaluru, India
harishl20050611@gmail.com

Abstract—In this paper, presents a new AI-based tutoring named as Smart Siksha. students educational application used by rural students. It increases personalization and assists them to learn. The students students who can learn out of this application are 8-12. The system contains modern Next.js web client, cross-platform flutter. client, and also a FastAPI backend that creates a customized based on their curriculum, it possesses a mock, AI tutor. generator of tests, paper setter, community services, and offline network which can synchronize the data when the device gets online. access. It is an effective architecture that incorporates low internet. access, multilingual, classes and as-specific to students. such sessments are a major addition. The Smart Siksha app has five almost languages—Hindi, Telugu, English, Kannada and Tamil. Student operationalized AI, 3,355 chapters, and 66 courses. tutoring LangChain with a Groq-hosted.

Index Terms—Smart Shiksha, AI in education, multilingual tutoring, offline-first systems, rural ed-tech, FastAPI, Next.js, Flutter, LangChain, LLaMA, learning analytics, formative assessment.

I. INTRODUCTION

The unfairness that occurs to the students when offered learning opportunities in these locations with low connectivity and fewer resources is due to the subsequent systemic impediments:

- 1) qualified teachers and resources as well as customized examinations.
- 2) poor internet accessibility.
- 3) The language barrier between learning material and the interests of the learners.

We have come up with an intelligent learning application known as Smart Siksha that assists the rural students by bringing these changes. We stand out among other tutoring platforms and content portable since our platform is created as a full-stack intelligent learning platform to which there are role-based workflows, social engagements, evaluation, and offline features. The Smart Siksha will be developed considering the attention to Indian education system where the board exam

format exists such as NCERT and other competitive exams such as IIT-JEE and NEET

The primary goals of Smart Shiksha are:

- 1) To offer personalized and context-aware tutoring responses utilizing a state-of-the-art LLM. The model utilizes LLaMA 3.3 70B..
- 2) It ensures the continuation of the students learning even though there is no internet connection by using the first offline architecture.
- 3) To offer the students an exam-based questions practice and evaluation of their answers in conformance to board syllabus.
- 4) To offer learner interfaces and AI responses in five different languages.
- 5) To offer a deployable architecture that can be locally developed and/or migrated to the cloud.

Here in this paper the Section II provides background and related work to place our design decisions, which helps to understand the project better. From sections III through IX we are going to show the system architecture, implementation, mathematical formulations, methodology, experimental results, and limitations of Smart Shiksha itself.

II. BACKGROUND AND RELATED WORK

A. Rural Education Challenges

Here in this paper the Section II provides background and related work to contextualize our design decisions. From sections III through IX we cover the system architecture, implementation, mathematical formulations, methodology, experimental results, and limitations of Smart Shiksha itself. [3], [4], [7]. In the urban centers, the teacher to student ratio has increased to 67:1, that is very high considering that UNESCO recommends 30:1. There is also a decreased number of teachers to students in the rural areas and the students are not equipped with modern learning tools and this brings about

a digital divide, as only 35 per cent of the students are well versed with basic computer skills yet many of the states have high usage of smart phones [1], [3]–[5]. The students in rural places are denied 70 percent more learning opportunities compared to the students in the urban areas [5], [7],] and the learning tools are only in Hindi and English languages which is hard to comprehend by students who do not speak the languages [3], [4].

B. Related AI Tutoring Systems

Several such systems have been developed to address the challenges such as **Mind Craft** [2] which uses the NCERT curriculum to fine-tune GPT-4 and analyze the performance over 10,000+ data points to achieve an improvement of 11 test scores. **Vidhya** [1] is an application which is designed to serve rural areas teenagers with multilingual support and it also uses deep learning to provide pronunciation feedback using Mel Frequency Cepstral Coefficients. **Shikshak** [3] is an authoring tool for developing an ITS system using fuzzy logic to adapt to the needs of individual learners. The scoring and mastery formulations in Section V of this paper are partially inspired by the concepts developed in this system. **Quest Genius** [5] uses OCR-based grading to achieve an accuracy of 89 evaluating board exam answers. Studies have shown the positive impact of AI-based tutoring on the outcome. It has been observed that the performance of students using an ITS system is better than those under regular tutoring [9], [10], with a score improvement of 5 to 20 the AI-based systems [2], [3], [5].

C. Gaps That Motivate Smart Shiksha

In spite of these results, existing systems have four important limitations that Smart Shiksha has been developed to address: (i) none of these systems is designed to work offline without the Internet, (ii) most of the existing systems support only one or two languages, (iii) none of the existing systems is a complete end-to-end system that supports all the four features of tutoring, testing, examination creation, and community support in a single deployable stack, and (iv) the problem of teacher onboarding and data privacy has remained untouched in all the existing systems [3], [4], [7], [9], [11]. The following sections explain how Smart Shiksha has addressed each of these gaps.

III. SYSTEM ARCHITECTURE

Smart Shiksha goes through a layered architecture with separate client surfaces, application services and AI inference components. The design directly addresses the four gaps identified in Section II.

A. Layered Architectural View

Figure 1 shows the overall architecture.

B. Runtime Dataflow for AI Tutoring

The primary data flow for interactive systems is the AI tutoring request data flow. This data flow consists of authentication requests, learner records, prompt creation, LLM inference, and message persistence, have been shown in Figure 2.

C. Deployment Topology

The platform is deployable for both local environments and cloud environments using containerized components with independently evolvable service boundaries, as presented in Figure 3.

IV. IMPLEMENTATION DETAILS

A. Frontend Subsystem

The subsystem of the web is also implemented in Next.js (App Router) using TypeScript. It offers dashboard, subject and AI tutor, mock-tests, exams-generator, community, text-books, offline-manager and administrator modules. Authentication is done with Clerk and all API requests are made via a typed Axios client module..

The main frontend features are:

- 1) Progressive Web App (PWA) installation with offline indicators.
- 2) Multilingual UI contexts and localization utilities for five Indian languages.
- 3) IndexedDB caching via Dexie for offline content continuity.
- 4) Route-level organization for learner and administrator workflows.

B. Backend Subsystem

The backend is implemented with the help of FastAPI and asynchronous SQLAlchemy patterns (MongoDB adapter support). The service boots with lifecycle-managed startup and shutdown hooks and mounts versioned API routers under, major API domains include:

Major API domains include:

- 1) Authentication and onboarding.
- 2) Curriculum content (subjects, chapters, topics).
- 3) AI tutor interactions and chat session management.
- 4) Mock tests and auto-evaluation.
- 5) AI-assisted exam paper generation and PDF output.
- 6) Progress tracking and dashboard metrics.
- 7) Community posts and replies.
- 8) Textbook upload and retrieval.
- 9) Offline bundle endpoints for lessons and quizzes.

C. AI Service Integration

LangChain is also used with ChatGrok to access a Groq hosted LLaMA 3.3 70B model in the Smart Shiksha system. The prompt has been customized to encourage tutoring training, formulate examination questions, clarify subject content, and break textbook paragraphs down. Features of the tutoring process, including the grade level, subject area, and topic of a student, their language, and chosen chat history are provided to the model to provide sufficient contextual information. When an API key is to be used, the system shows an easily understandable notification to maintain the user experience. This is particularly relevant to the rural schools that might experience difficulties in the first implementation. It is important that the application does not crash or give out error messages with technical meanings to the end user.

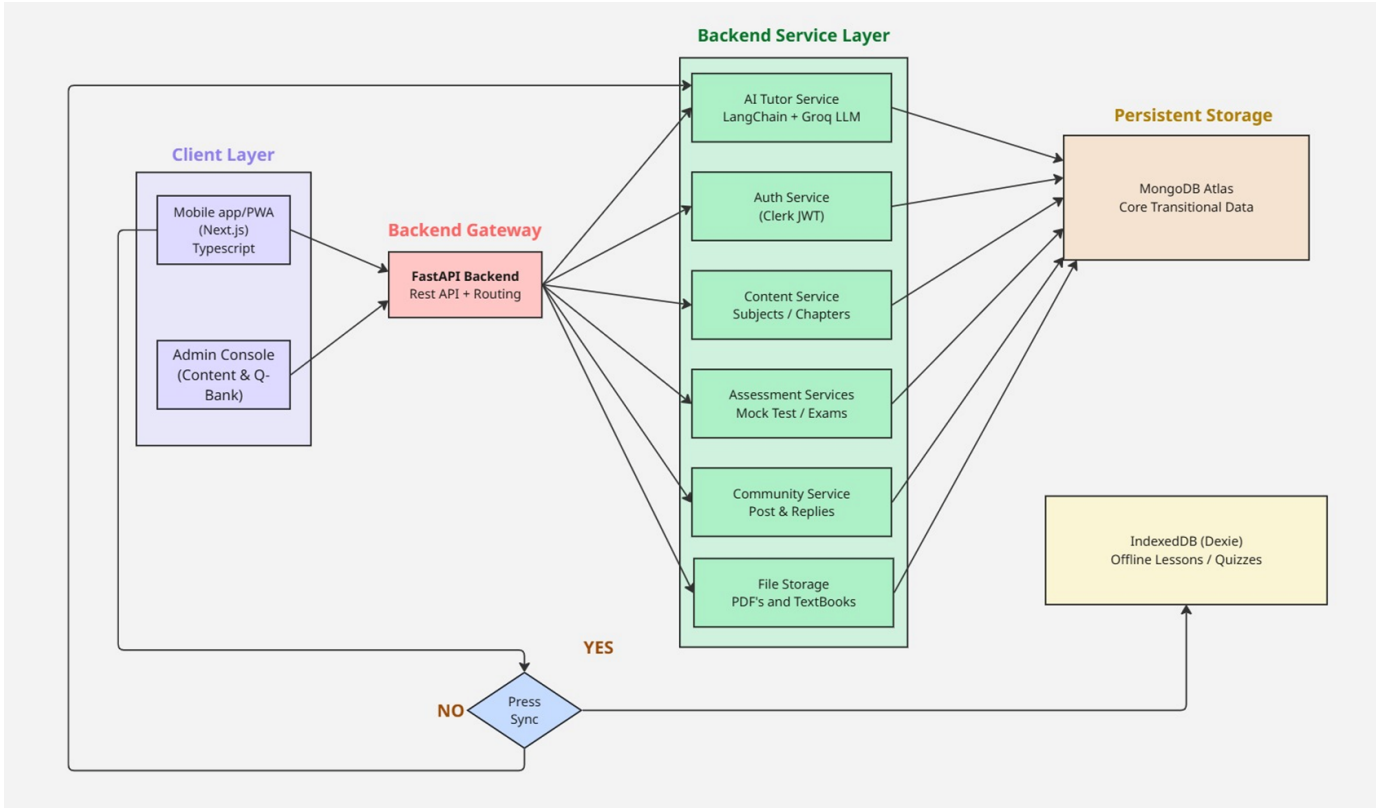


Fig. 1. Architecture

D. Data Model and Persistence

These organizations are users, subject, chapter, topic, questions, mock exams, test questions, student attempts, progress, chat, chat messaging, text books, examination papers and community posts. The arrangement enables multi-statefulness with four correlated structures of the graph.

This supports multi-feature statefulness organized into four graphs:

- 1) Pedagogical graph: subject → chapter → topic.
- 2) Assessment graph: test → questions → attempts.
- 3) Conversational graph: sessions → messages.
- 4) Social graph: posts → replies through parent references.

E. User Input and Query Processing

This explains how the system handles the user's input from start to finish. **User Input (Question/Query Data):** This is the starting point of the whole process. The user provides a question or query, which initially comes in an unstructured form.

Data Preprocessing (Cleaning, Tokenization): The raw input is cleaned by making sure that it does not contain any stop words, punctuations, or special characters and correcting any errors. The purged text is then tokenized - divided into smaller units known as tokens - and thus rendered easier to analyze by the computer.

Comparative Analysis (Keyword Match, NLP, AI): At first the simplest keyword match is attempted. Then the

meaning and context of the query, word relationships, and intent of the user are under-stood using NLP. Lastly, ML models that are trained on massive datasets accomplish more complicated tasks such as detecting semantic correlations and query types.

Result Generation (Answer Display / Output): Anyone in the top-ranking is formatted and displayed to the user in form of a direct answer or explanation.

V. MATHEMATICAL FORMULATIONS

This section will explain the various mathematical equations that are implemented in the system for the shikshak scoring model as well as the Smart Shiksha mastery tracking model.

A. Response Time Score Function

The first phase of the score for each question is a combination of the response time t and the correctness c with respect to a threshold value t_r defined by the teacher. This is represented as:

$$s_1(c, t) = \begin{cases} c, & t \leq t_r, \\ c \frac{3t_r - t}{2t_r}, & t_r < t \leq 3t_r, \\ 0, & t > 3t_r, \end{cases} \quad (1)$$

Where c is the correctness of the response. This will give full credit for the response as long as the response is within the threshold time t_r . The credit will linearly decay until $3t_r$

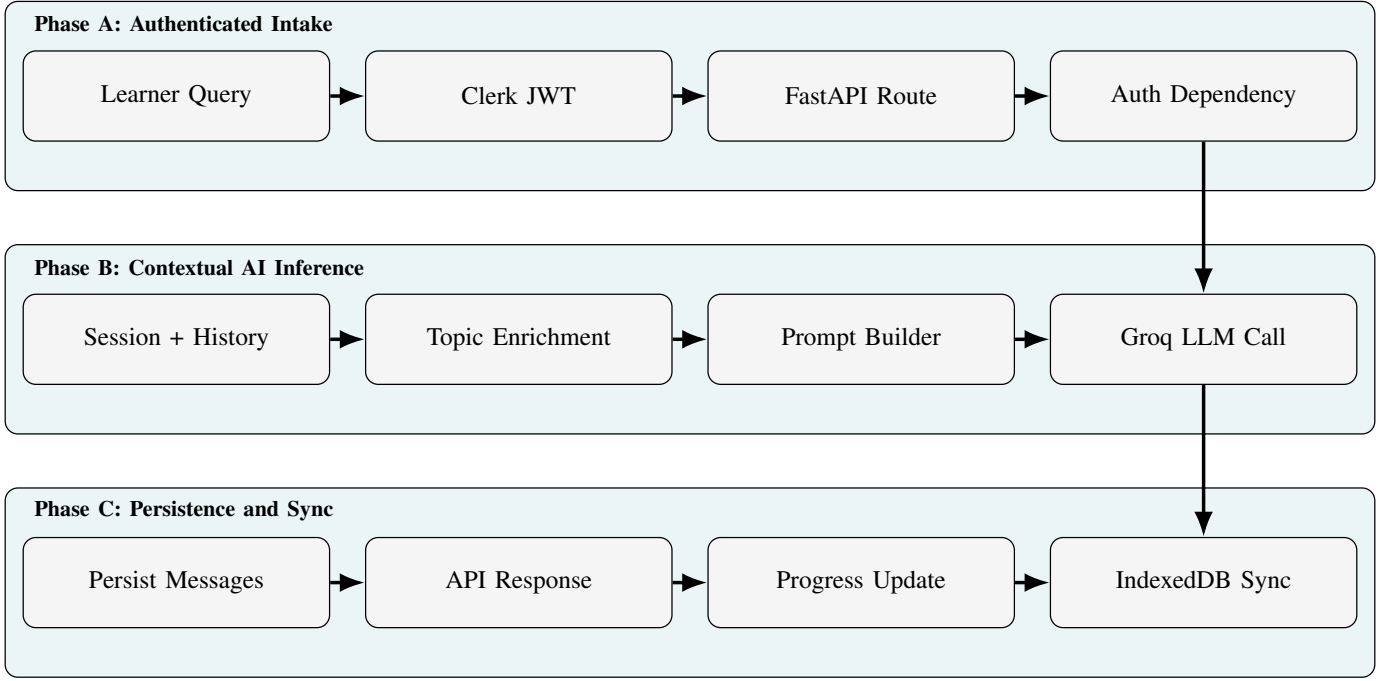


Fig. 2. End-to-end tutoring flow from client query to response persistence and offline synchronization.

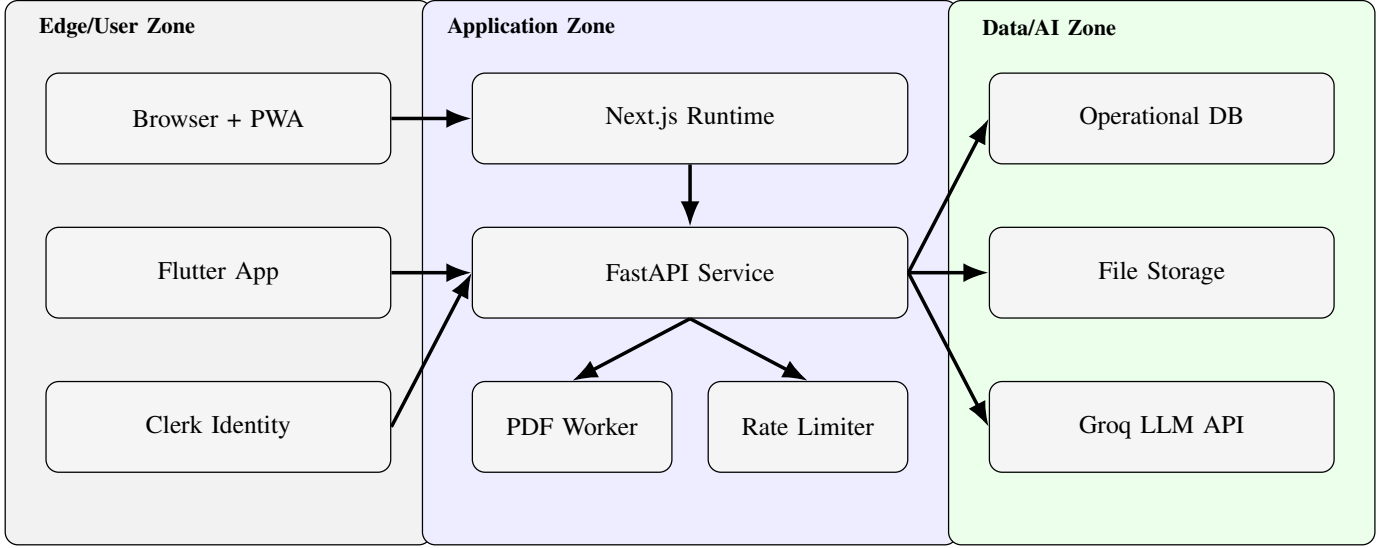


Fig. 3. Hybrid deployment topology showing edge clients, application runtime, data stores, and external AI endpoint.

is reached. After that time, the response will be zero. This is important because the speed as well as the correctness of the response will show how well the student has understood the topic.

B. Difficulty and Relevance Weighted Score

The final score for the question is a combination of the difficulty $diff$ and the relevance rel that is also defined by the teacher. This is represented as:

$$s = s_1(c, t) \times diff \times rel \quad (2)$$

This will give the student a greater reward for answering a question that is difficult. The score s is the result that is sent to the students' state machine.

C. State-Transition Function

Shikshak models each student's composite state S by coverage and a fuzzy linguistic performance term. State transitions happen only if the input exceeds the topic specific threshold i_Q :

$$S_{next} = \begin{cases} \delta(S_{current}, i), & i \geq i_Q, \\ S_{current}, & \text{otherwise} \end{cases} \quad (3)$$

where δ is the transition function. This ensures that topics advance only when the student has demonstrated sufficient level of mastery.

D. Fuzzy Membership Function for Performance States

The students performance is dericed from two normalized continuous metrics, that are: comprehension C and problem-solving P , both in $[0, 1]$. The system defines membership functions for five linguistic labels: Excellent, Very Good, Good, Average, and Poor. For example, the “Good” membership is defined as:

$$\mu_{Good}(C, P) = \max[\min(C, 2P - 1), \min(2C - 1, P)] \quad (4)$$

After each test, the pair (C, P) is fuzzified to assign the student to one of the five performance states.

E. Aggregate Comprehension and Problem-Solving Metrics

$$C_{topic} = \frac{1}{m} \sum_{j=1}^m \alpha_j^C s_j, \quad P_{topic} = \frac{1}{m} \sum_{j=1}^m \alpha_j^P s_j \quad (5)$$

Here m is the number of questions in the test, s_j is the final weighted score for question j , and α_j^C, α_j^P indicate whether question j measures comprehension or problem-solving respectively. This is used to avoid abrupt changes in the learner model, Shikshak applies a simple method that averages the new topics scores with the previous scores reducing sensitivity.

F. Exponential Moving Average of Metrics

To avoid abrupt changes in the student model, Shikshak smooths topic-level metrics with historical values:

$$C_{new} = \frac{C_{old} + C_{topic}}{2}, \quad P_{new} = \frac{P_{old} + P_{topic}}{2} \quad (6)$$

G. Smart Shiksha Mastery Tracking (EMA)

For offline first mastery system the smart shiksha uses a method called as EMA (Exponential Moving Average). We chose this formula because it is simple enough to compute on a mobile device without an internet connection:

$$M_t = \alpha S_t + (1 - \alpha) M_{t-1} \quad (7)$$

Where:

- M_t : mastery estimate after test t
- S_t : score of the current test(0-1)
- M_{t-1} : previous mastery
- α : learning rate (we use $\alpha = 0.3$)

This formula is best for smooth and responsive tracking of student mastery in a subject for personalized curriculum tailoring while remaining simple enough for mobile deployment in low resource environment.

Table I compares the Smart Shiksha formula with the Shikshak approach:

TABLE I
COMPARISON BETWEEN SMART SHIKSHA AND SHIKSHAK FORMULAS

Aspect	Smart Shiksha (EMA)	Shikshak
Core formula	$M_t = \alpha S_t + (1 - \alpha) M_{t-1}$	$s_1(c, t) \times diff \times rel$ plus fuzzy state
Purpose	Continuous mastery estimation	Scoring correctness and response speed
Complexity	Simple weighted average	Piecewise functions with fuzzy logic
Offline Friendliness	Highly lightweight, ideal for mobile	Computationally heavier

VI. METHODOLOGY

A. Design Approach

The platform was built using a feature-domain decomposition strategy. We separate the modules by educational function (tutoring, assessment, community, offline) and then unify them by using a common authentication and persistence layer. This made it much easier for our team to work on different parts in parallel.

B. Offline-First Strategy

Offline functionality is implemented through two mechanisms:

- 1) Backend endpoints that export structured lesson and quiz bundles.
- 2) Client-side IndexedDB stores for durable local access via the Dexie library.

This approach works properly because the UI works independently from the network and cache refresh happens in the background, which separates the synchronization concerns from rendering concerns.

C. Evaluation Protocol

We validated the system at two levels. First, we validated the technical build:

- 1) Backend compile and service startup checks.
- 2) Frontend production build checks (Next.js).
- 3) Flutter static analysis and Windows release build.
- 4) Backend latency sampling on the health endpoint.

Second, we checked the system against the literature findings. For example, we made sure our offline design addressed the connectivity issues that [3], [4], [7] identified, and our multilingual support addressed the language barriers that [1], [6] highlighted.

VII. EXPERIMENTAL RESULTS AND OBSERVATIONS

A. Build and Runtime Status

Local validation shows successful operation across all three major surfaces:

- 1) Backend service starts and passes health checks.
- 2) Next.js production build completes successfully.
- 3) Flutter analysis and Windows build succeed.

B. Latency Measurement

For local loopback testing on GET /health ($N = 15$ samples), we got the following results:

- Mean latency: $\mu = 4.80$ ms.
- P95 latency: 5.63 ms.

The mean was computed as:

$$\mu = \frac{1}{N} \sum_{i=1}^N t_i \quad (8)$$

where t_i is the per-request latency and $N = 15$.

These numbers show very low transport and handler overhead, which gives us confidence that the interactive tutoring flows will feel responsive to students even on modest hardware.

C. Comparison with Related Work

Table II shows how our work compares with the research papers we reviewed in the literature survey.

TABLE II
COMPARATIVE ANALYSIS OF AI IN EDUCATION APPROACHES

Paper/System	Focus Area	Methodology	Key Contribution	Main Challenge
AI for Rural Education [8]	Review	Literature analysis	Overview of AI potential	Infrastructure gaps
Vidhya [1]	English tutoring	Voice/text AI	Multi-level feedback	Limited connectivity
Shikshak [3]	ITS authoring	Fuzzy logic scoring	Adaptive tutoring	Teacher shortages
Quest Genius [5]	Secondary ed.	Adaptive AI	Personalized learning	Data privacy
Smart Shiksha (Ours)	Full-stack platform	LLM + offline sync	End-to-end rural ed-tech	Production scale

VIII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

The limitations of our current work are:

- 1) The latency is gathered from local environment testing and cannot represent real time internet load behaviour.
- 2) We do not yet have actual learning outcomes like scores improvement over a semester for real students, as we have just deployed our app.
- 3) The quality of the AI response can vary by the behavior of the prompt and the model. We have not yet added a strict factual verification layer, which is something we plan to add.
- 4) AI systems can sometimes too narrowly focus on content tasks and miss creativity or critical thinking aspects, as our literature review also noted [9], [11].
- 5) Teacher training is still a gap the system is there, but we need a proper onboarding program for rural educators who are comfortable with technology.

B. Future Work

Based on both our literature survey and our implementation experience, we have the following plans:

- 1) Introduce retrieval-augmented generation (RAG) from board-verified NCERT corpora so the AI tutor gives more accurate, curriculum-aligned answers.
- 2) Add automated test suites across API contracts, route rendering, and mobile startup workflows.
- 3) Implement adaptive recommendation policies based on long-term learner performance data.
- 4) Expand observability with structured telemetry and per-feature service-level objectives.
- 5) Conduct field studies in rural schools to measure actual educational impact, student retention, and language accessibility outcomes.
- 6) Explore VR/AR modules for subjects like science and geography where visualization is important.
- 7) Build a mixed AI-human model where teachers can monitor AI interactions and jump in when needed [5], [11].

IX. CONCLUSION

At this stage, Smart Shiksha demonstrates that an AI-based education framework can be developed to address the constraints of rural education systems - interrupted connections, multilingual interactions, and test-based procedures. The framework provides a deployable reference architecture which combines LLM-based tutoring and effective software engineering practices - modular APIs, structured persistence, offline synchronization, and cross-platform access via Next.js web and Flutter mobile clients.

As already illustrated in the background survey in Section II, the most significant untapped gaps in the existing systems are as follows: AI tutoring improves student performance and the most valuable gaps are end-to-end workflow coverage, multilingual support and offline access. Smart Shiksha is concerned with all three in particular. Technical viability has been confirmed currently, with an average latency of 4.80 ms and builds on all target platforms. The future work will be concerned with field deployments in rural schools to evaluate actual learning outcomes, introduction of retrieval-augmented generation by board-verified NCERT corpora, and robustness on production scale.

This model can also be implement in rural schools using a sing computer as its server which has a local LLaMa model installed and give LLM access to its students without needing internet.

ACKNOWLEDGMENT

We thank Dr. C. Viji for guidance throughout this project. We also acknowledge the contributors to the open-source frameworks used in Smart Shiksha, including FastAPI, Next.js, Flutter, SQLAlchemy, LangChain, and associated ecosystem maintainers.

REFERENCES

- [1] S. Anand, K. A. Reddy, P. A. Reddy, C. S. Srinivas, and S. N. Rao, "Vidhya: A virtual tutor for enhancing English education in rural areas of India," Amrita Vishwa Vidyapeetham, 2025.
- [2] A. Bardia and A. Agrawal, "MindCraft: AI-powered personalized education for rural India," arXiv preprint, vol. 2502.05826v1, 2025.
- [3] S. Chakraborty, T. Bhattacharya, P. K. Bhowmick, A. Basu, and S. Sarkar, "Shikshak: An intelligent tutoring system authoring tool for rural education," *J. Educ. Technol.*, vol. 12, no. 1, pp. 45–62, 2025.
- [4] A. Tripathi, V. Yadav, and S. Kumar, "Leveraging artificial intelligence for rural education: A systematic review of transforming learning opportunities and bridging the urban-rural divide," Preprints.org, 2025.
- [5] V. Prakash, R. Bansal, and S. Sharma, "Quest-Genius: An AI-driven personalised learning platform bridging educational gaps for secondary students," in *2025 IEEE Int. Conf. Educ. Data Mining Poster*, 2025.
- [6] M. Jeyakumaran, P. Saravanan, and K. Sundararajan, "Revolutionizing education: The impact of artificial intelligence on personalized learning and teacher roles in India," *Int. J. Sci. Manag. Stud.*, vol. 8, no. 1, pp. 100–115, 2025.
- [7] O. P. Mokoena and S. M. Seeletse, "AI in rural classrooms: Challenges and perspectives from South African educators," *Int. J. Curr. Educ. Stud.*, 2025.
- [8] V. Yadav, A. Tripathi, and S. Kumar, "AI for rural education: Opportunities and challenges," *J. Adv. Artif. Intell. Eng. Technol.*, vol. 1, no. 2, pp. 11–29, 2025.
- [9] H. Zhang, Y. Li, and X. Chen, "AI in intelligent tutoring systems for sustainable education: A systematic review," *Sustain. Comput.: Inform. Syst.*, vol. 31, pp. 100–117, 2023.
- [10] I. Mounkoro et al., "AI-powered tutoring systems: Revolutionizing individualized support for learners," *Library Prog. Int.*, vol. 43, no. 2, pp. 344–360, 2023.
- [11] J. Wang, L. Xu, and Z. Zhao, "AI and personalized learning: Bridging education and technology," *Int. J. Educ. Technol.*, vol. 15, no. 4, pp. 200–223, 2024.
- [12] R. Gupta and P. Singh, "AI's potential in transforming Indian education," *Int. J. Educ. Dev. Using ICT*, vol. 18, no. 3, pp. 50–68, 2022.
- [13] S. Patel and R. Mehta, "AI-powered e-learning platform: Role-based access, intelligent recommendations, and real-time query resolution," *E-Learning Innov. J.*, vol. 5, no. 2, pp. 70–89, 2024.
- [14] X. Liu and N. Sharma, "Personalized education and AI in US, China, and India: A systematic review," *Comput. Educ.*, vol. 178, p. 104379, 2021.
- [15] FastAPI, "FastAPI Framework Documentation," 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [16] Next.js, "Next.js Documentation," 2026. [Online]. Available: <https://nextjs.org/docs>
- [17] Flutter, "Flutter Documentation," 2026. [Online]. Available: <https://docs.flutter.dev/>
- [18] LangChain, "LangChain Documentation," 2026. [Online]. Available: <https://python.langchain.com/docs/>
- [19] Groq, "Groq API Documentation," 2026. [Online]. Available: <https://console.groq.com/docs>